

Food identification with segmentation

Just as an engine requires high-quality fuel for optimal performance and long-term maintenance, humans require nutritious food to enhance their health and overall quality of life. This necessity becomes even more critical with aging or for individuals requiring specialised care. Therefore, the development of an intelligent system capable of accurately segmenting and recognising food items on a plate would be highly beneficial, particularly in clinical settings where monitoring patients' dietary intake is essential.

Semantic image segmentation is the process of classifying each pixel into a predefined category and assigning a label to every pixel, for our dataset the algorithm classifies each pixel in an image into one of 104 categories (103 food classes plus 1 background class), thereby segmenting the image into distinct regions (e.g., "background" and "foreground" or specific objects such as "carrot" and "banana").

To implement image segmentation, a U-Net-based architecture was designed to accurately segment food items on plates.

The structure of the architecture:

1. **Encoder:** The encoder downsamples the input image to extract features at multiple scales
 - Two convolutional layers (Conv2D layers (3x3 convolutions)) with increasing filter sizes.
 - BatchNormalization for stabilising and accelerating training.
 - MaxPooling2D to reduce spatial dimensions (downsampling).
2. **Bottleneck:**
 - Using a deep convolutional layer. The deepest part of the network (16x16x1024), where the most compressed feature representation exists and captures high-level features. No pooling here, this is the bottom of the U.
3. **Decoder:** The decoder upsamples the features back to the original resolution, combining them with skip connections from the encoder to preserve spatial details.
 - UpSampling2D: Uses upsampling to restore spatial resolution.
 - **Concatenate:** Combines the upsampled features (u6) with the corresponding encoder features (c4, 32x32x512) via a skip connection.
 - Incorporates skip connections that help recover fine details lost during downsampling.
 - Two convolution layers refine the features.
4. **Final Output:**
 - A 1x1 convolution layer to reduce the 64-channel feature map to 104 channels (number of classes)
 - Applies softmax to produce a probability distribution over classes for each pixel.
5. **Evaluation:**
 - Mean Intersection over Union (Mean IoU)

Encoder:

The encoder is responsible for extracting features from the input image. It progressively reduces the spatial dimensions while capturing complex and abstract features.

Bottleneck:

The bottleneck serves as the core of the architecture, linking the encoder and decoder. It processes the most abstract representation of the input image, extracting high-level features crucial for precise segmentation.

Decoder:

The decoder forms the second half of the architecture, responsible for upsampling the feature maps from the bottleneck and generating the final segmentation mask. It mirrors the encoder's structure but operates in reverse, progressively increasing the spatial dimensions while refining the output.

Final Output

At the end of the decoder, a 1x1 convolutional layer transforms the final feature maps into a segmentation mask. The number of filters corresponds to the number of classes(104 for our dataset), and a softmax activation function generates a probability map for each pixel. The pixel's label in the final output is determined by the class with the highest probability.

Evaluation

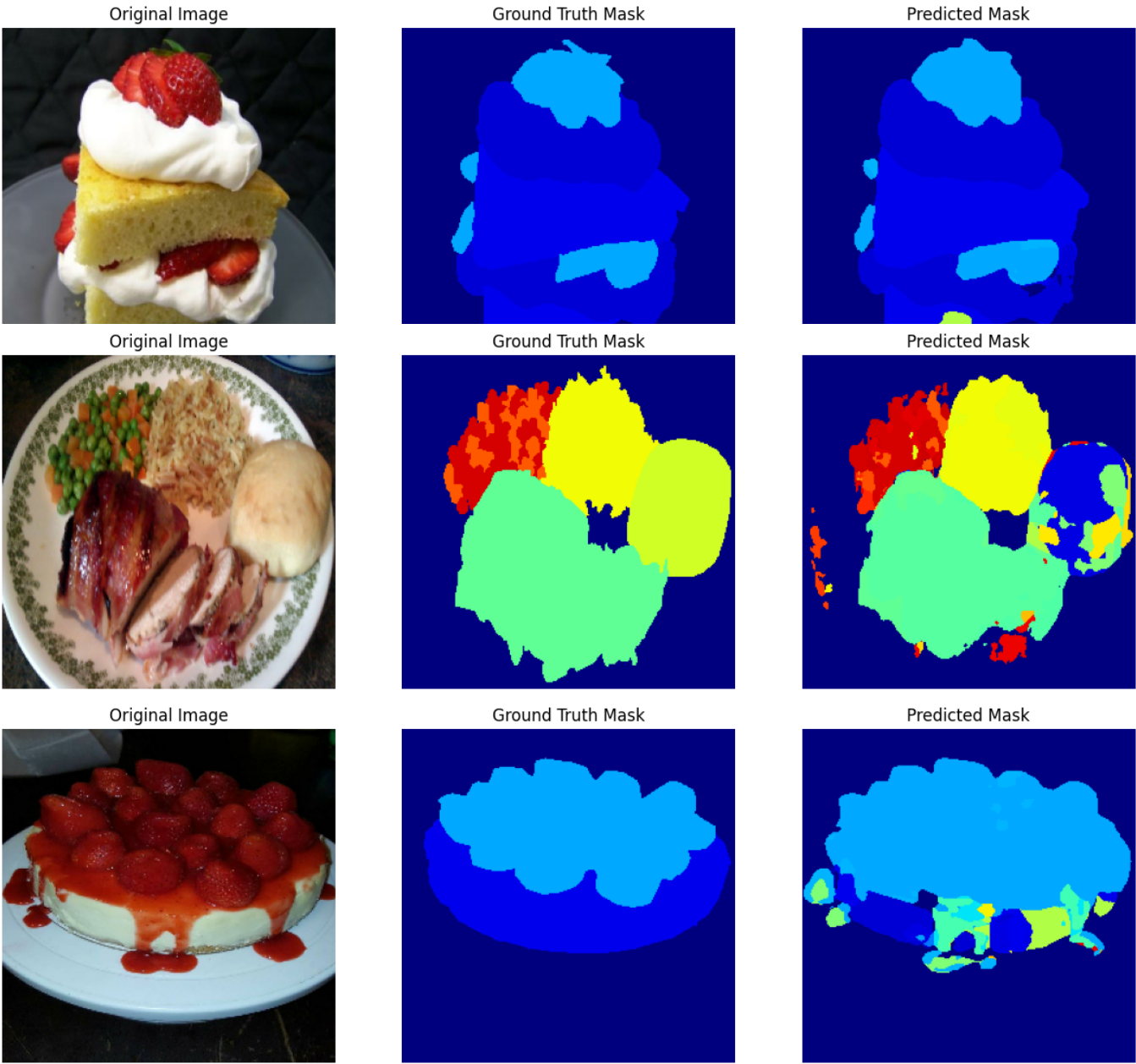
The Mean Intersection over Union (**Mean IoU**) was used to evaluate the performance of the unet-based semantic segmentation model. Mean IoU is a standard metric for segmentation tasks, measuring the overlap between predicted and ground-truth segmentations ranging from 0 (no overlap) to 1 (perfect overlap).

The default dataset for this task originally included two separate sets for training and testing. However, I treated the test set as a validation set and further separated a fraction of it for the actual test set to evaluate the model's performance. This approach not only assesses the model's accuracy but also allows for testing on entirely new images to observe how well the model generates segmentations.

Here, detailed information about the model's performance using the IoU metric can be found:

Model's Performance	
Number of epoch	70
Training data Accuracy	80%
Validation data Accuracy	67%
IoU training data	0.3
IoU Validation data	0.16
IoU Test data	0.17

Here are the output segmented images generated by the model from the test dataset

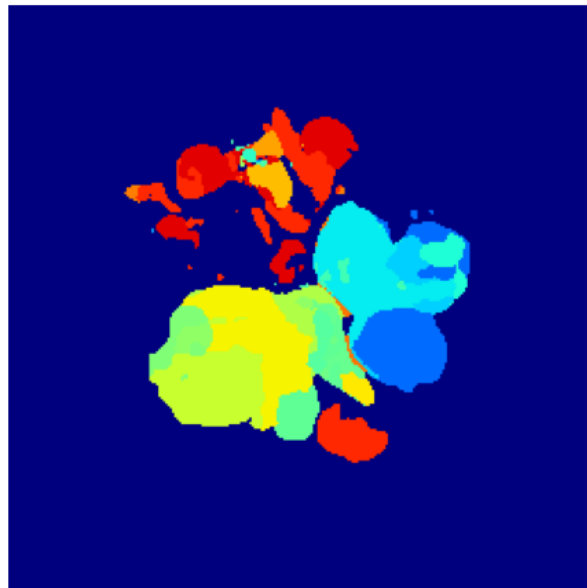


Here is the output of a newly segmented image that I provided to the model:

Original Image



Segmentation Mask



Here is the output segmented and labeled by model on the test dataset :

Original Image



Ground Truth Mask



Predicted Mask with Labels



Approaches employed to improve my network and model:

- Adding BatchNormalization layers to the convolutional layers in the encoder, decoder, and bottleneck sections to stabilize and accelerate training.
- Adding Dropout layers to prevent overfitting and improve the model's generalization ability.
- Reducing the learning rate to achieve more stable convergence.

The Flowchart of the Algorithm

